

DIGITAL NOTES

ON

WEB TECHNOLOGIES

III B.TECH – I SEM



● **DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**
Sreyas Institute of Engineering and Technology
An Autonomous Institution

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC-A Grade, NBA (CSE, ECE & ME) & ISO 9001:2015 Certified
Bandlaguda, Nagole, Hyderabad, Telangana 500068

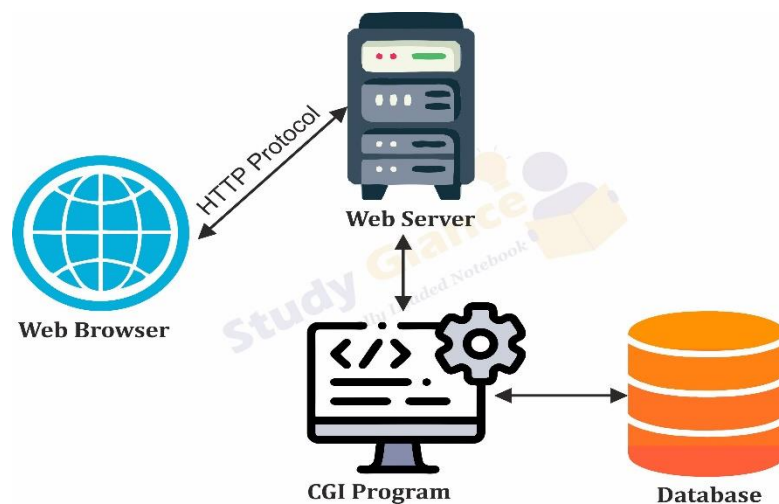
Unit 3

Introduction to Servlets: Common Gateway Interface (CGI), Life cycle of a Servlet, deploying a servlet, The Servlet API, Reading Servlet parameters, Reading Initialization parameters, Handling Http Request & Responses, Using Cookies and Sessions, connecting to a database using JDBC.

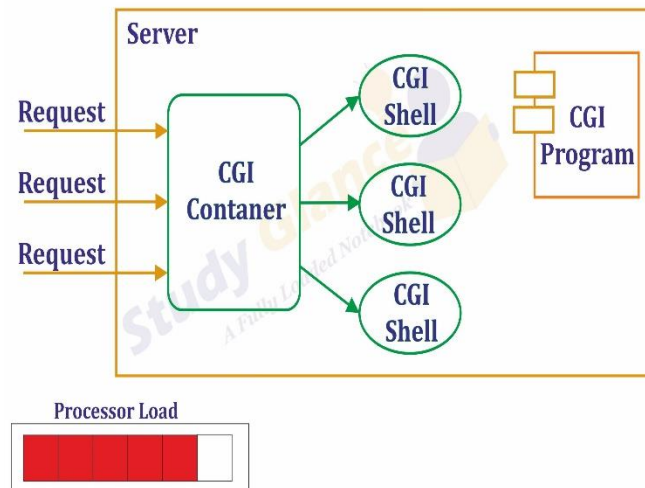
COMMON GATEWAY INTERFACE

The Common Gateway Interface (CGI) is the standard process that uses a set of rules to propagate the user's request to the web resources such as web server or web application program and respond to the user through the web interface. CGI includes several working scripts and programs for web communication. CGI is the mechanism that is part of the Hypertext Transport Protocol (HTTP). One of the examples of CGI flow is the Web browsers send the forms data to the backend server, and CGI connects to the application program on the web-server and the program response to the web browser.

CGI Architecture



- **Common Gateway Interface (CGI)**
- The Common Gateway Interface (CGI) is the standard process that uses a set of rules to propagate the user's request to the web resources
- CGI includes several working scripts and programs for web communication.
- CGI is the mechanism that is part of the Hypertext Transport Protocol (HTTP).
- One of the examples of CGI flow is the Web browsers send the forms data to the backend server, and CGI connects to the application program on the web-server and the program response to the web browser.



- The Common Gateway Interface (CGI) provides the middleware between servers and external databases or information sources.
- The Web server typically passes the form information to a small application program that processes the data and may send back a confirmation message.
- This process or convention for passing data back and forth between the server and the application is called the common gateway interface.
- The following image describes how a web server acts as an intermediate between the CGI program and the client browser.

Features of CGI

- It is a very well defined and supported standard.
- CGI scripts are generally written in either Perl, C, or maybe just a simple shell script.
- CGI is a technology that interfaces with HTML.
- CGI is the best method to create a counter because it is currently the quickest
- CGI standard is generally the most compatible with today's browsers

Advantages of CGI

- The advanced tasks are currently a lot easier to perform in CGI than in Java.
- It is always easier to use the code already written than to write your own.
- CGI specifies that the programs can be written in any language, and on any platform, as long as they conform to the specification.
- CGI-based counters and CGI code to perform simple tasks are available in plenty.

Disadvantages of CGI

- In Common Gateway Interface each page load incurs overhead by having to load the programs into memory.
- Generally, data cannot be easily cached in memory between page loads.
- There is a huge existing code base, much of it in Perl.
- CGI uses up a lot of processing time.

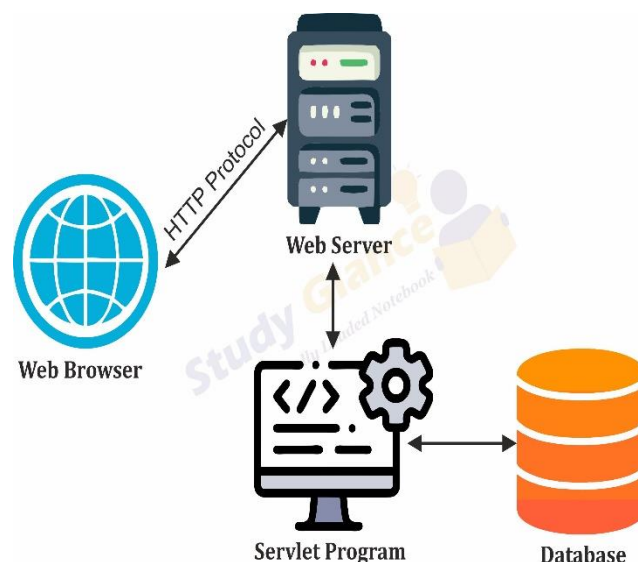
SERVLETS

- Servlets are small programs that execute on the server side of a Web connection and dynamically extend the functionality of a Web server just as applets are used to create a web application.
- Servlets are protocol and platform independent server-side components, written in Java and executes within JVM of web server, in much the same way that applets are loaded and executed within the JVM of the Web client.
- Servlet is an API that provides many interfaces and classes including documentation.
- Servlet is a web component that is deployed on the server to create a dynamic web page.
- Since servlets run inside the servers, however, they do not need a graphical user interface (GUI). In this sense, servlets are also faceless objects.
- Servlets offer a number of advantages over other approaches, including: portability, power, efficiency, endurance, safety, elegance, integration, extensibility, and flexibility.

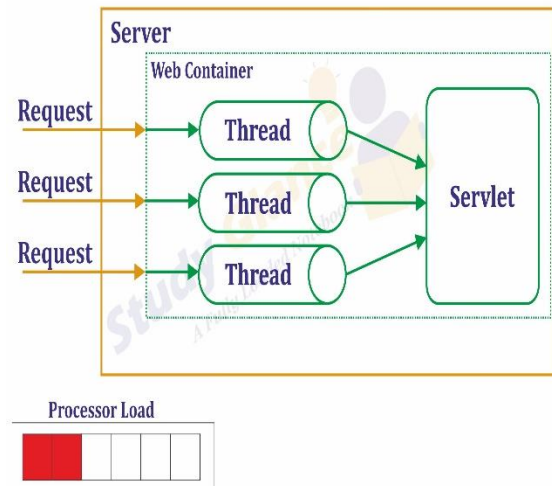
Servlet Architecture

The architecture, is the communication interface, Basically, it performs the below-mentioned tasks.

- The clients send the request to the webserver.
- The web server receives the request.
- The web server passes the request to the corresponding servlet.
- The servlet processes the request and generates the response in the form of output.
- The servlet sends the response back to the webserver.
- The web server sends the response back to the client and the client browser displays it on the screen.



- A new process is started for each client request. The creation of the process for every such request requires time and significant which limits the number of requests a server can handle
- Servlets are thread based applications and work on server level and they share data among each other.
- All the programs of Servlets are written in JAVA and they get to run on JAVA Virtual Machine. The following image describes how a request from clients is served with the help of threads:



DIFFERENCE BETWEEN CGI AND SERVLETS

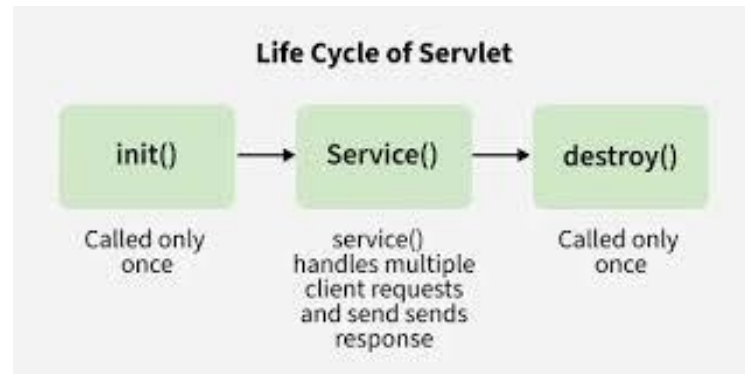
CGI	Servlet
The CGI programs are written in native OS.	Servlet is written in java class & its runs in JVM.
CGI creates a process base for each request.	Servlet creates a new thread to process each request.
The CGI is a language-independent interface that allows a server to start an external process.	The main purpose of servlets is to add up the functionality to a web server.
A CGI program needs to be loaded and started on each CGI request.	Servlets stay in the memory while serving the requests.
CGI is not able to read the HTTP servers.	Servlets are useful to read and set the HTTP servers.
CGI are platform dependent.	Servlets are platform independent.
CGI is less secure and is not useful for data sharing.	Servlets are more secure and useful for Data sharing.

THE LIFE CYCLE OF A SERVLET

Three methods are central to the life cycle of a servlet. These are **init()**, **service()**, and **destroy()**.

They are implemented by every servlet and are invoked at specific times by the server.

Let us consider a typical user scenario to understand when these methods are called.



- First, assume that a user enters a Uniform Resource Locator (URL) to a Web browser. The browser then generates an HTTP request for this URL. This request is then sent to the appropriate server.
- Second, this HTTP request is received by the Web server. The server maps this request to a particular servlet. The servlet is dynamically retrieved and loaded into the address space of the server.
- Third, the server invokes the `init()` method of the servlet. This method is invoked only when the servlet is first loaded into memory. It is possible to pass initialization parameters to the servlet so it may configure itself.
- Fourth, the server invokes the `service()` method of the servlet. This method is called to process the HTTP request. You will see that it is possible for the servlet to read data that has been provided in the HTTP request. It may also formulate an HTTP response for the client.
- The servlet remains in the server's address space and is available to process any other HTTP requests received from clients. The `service()` method is called for each HTTP request.
- Finally, the server may decide to unload the servlet from its memory. The server calls the `destroy()` method to relinquish any resources.

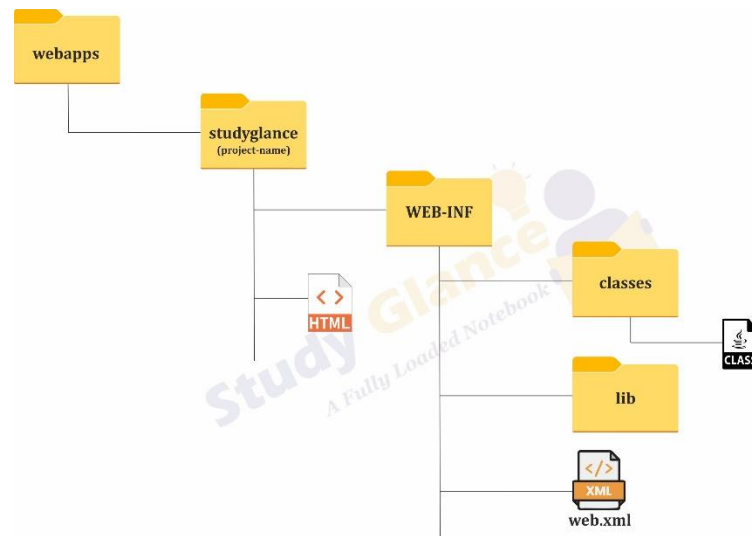
DEPLOYING A SERVLET PROGRAM

Steps for creating and running a servlet

1. Create a directory structure
2. Create a Servlet
3. Compile the Servlet
4. Create a deployment descriptor
5. Start the server
6. Access the servlet

(1) Create a directory structure

The directory structure defines that where to put the different types of files so that web container may get the information and respond to the client.



(2) Create a Servlet

The servlet example can be created by three ways:

- 1) By implementing Servlet interface,
- 2) By inheriting GenericServlet class, (or)
- 3) By inheriting HttpServlet class

NOTE: The mostly used approach is by extending HttpServlet because it provides http request *HelloWorld.java*

// Import required java libraries

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

// Extend HttpServlet class

```
public class HelloWorld extends HttpServlet {
```

```
    public void doGet(HttpServletRequest request, HttpServletResponse response)
```

```
        throws ServletException, IOException {
```

```
        // Set response content type
```

```
        response.setContentType("text/html");
```

```
        // Actual logic goes here.
```

```
        PrintWriter out = response.getWriter();
```

```

        out.println("<h1>" + message + "</h1>");
    }
}

```

Note: HttpServlet contains all the life cycle methods of Servlet and the service() method is written in the following two methods, they are

public void doGet (HttpServletRequest, HttpServletResponse) throws ServletException, IOException

public void doPost (HttpServletRequest, HttpServletResponse) throws ServletException, IOException

(3)Compile the Servlet

To compile the servlet program we have to set a CLASSPATH
 ..\tomcat\lib\servlet-api.jar

Now compile the HelloWorld.java using following command
 javac HelloWorld.java

If the compilation is successfull then HelloWorld.class file will be generated. Copy *.class file into document **root/WEB-INF/classes** folder

(4)Create a deployment descriptor

1. Whenever client makes a request to a servlet that request is received by server and server goes to a predefined file called web.xml for the details about a servlet.
2. web.xml file always gives the details of the servlets which are available in the server.
3. If the requested servlet is available in web.xml then server will go to the servlet, executes the servlet and gives response back to client.
4. If the server is not able to find the requested servlet by the client then server generates an error (resource not found).

web.xml

<web-app>

<servlet>

```
<servlet-name>demo</servlet-name>  
<servlet-class>HelloWorld</servlet-class>  
</servlet>  
<servlet-mapping>  
  <servlet-name>demo</servlet-name>  
  <url-pattern>/servletdemo</url-pattern>  
</servlet-mapping>  
</web-app>
```

Start the server

To start Apache Tomcat server, double click on the startup.bat available at **../tomcat/bin** directory.

Access the servlet

Open browser and write **http://hostname:portno/contextroot/url-pattern**. For example:

http://localhost:8080/studyglance/servletdemo